

Resolución de Incidente: Expiración de Sesión

Equipo: CSS | Plataforma: KiosQ | Estado: Verificado / No Bug

1. Descripción del problema

Durante las pruebas funcionales realizadas en la plataforma KiosQ, se evaluó el comportamiento del sistema ante la expiración de la sesión del usuario. El objetivo fue verificar de manera rigurosa la respuesta de la aplicación cuando un usuario intenta realizar peticiones a recursos protegidos con una sesión inválida o previamente expirada.

La prueba consistió en la remoción deliberada de la sesión activa en el cliente para posteriormente forzar una recarga del aplicativo, permitiendo observar el comportamiento coordinado entre el frontend y el backend durante el proceso de autenticación.

Como resultado del flujo esperado de protección, el servidor rechazó las solicitudes dirigidas a endpoints restringidos respondiendo con el código de estado HTTP 401 (Unauthorized). Específicamente, al invocar el endpoint `/api/me`, se registraron las siguientes trazas de salida en la consola de depuración:

```
Failed to load resource: the server responded with a status of 401 (Unauthorized)
[API Error]
[Auth] Session expired detected
```

2. Objetivo de la prueba

Verificar que el sistema reaccione de forma correcta, segura y predecible ante la pérdida de autenticación de un usuario, garantizando la hermeticidad de los recursos restringidos del sistema y previniendo cualquier tipo de acceso no autorizado.

3. Procedimiento realizado

La evaluación se llevó a cabo siguiendo una secuencia ordenada de pasos técnicos:

1. Despliegue local del entorno Backend desarrollado bajo el ecosistema **NestJS**.
2. Despliegue local del entorno Frontend estructurado en **React + Vite**.
3. Validación del flujo inicial y correcto funcionamiento de la capa de autenticación mediante la integración con **WorkOS**.
4. Acceso exitoso al módulo del **Dashboard** principal.
5. Eliminación manual y controlada del token/sesión activa desde las herramientas de desarrollo del navegador.
6. Ejecución de una recarga forzada (Reload) de la aplicación web.

7. Monitoreo, captura y análisis de las peticiones HTTP concurrentes desde la pestaña *Network* de las herramientas de desarrollo.
8. Inspección de excepciones y logs de salida generados en la consola del navegador.
9. Auditoría estática del flujo lógico de autenticación directamente en el código fuente del proyecto.

4. Análisis del Comportamiento en Red y Consola

Al procesar la pérdida de sesión, la capa de red del navegador interceptó la denegación explícita de acceso por parte del servidor API a través de los siguientes códigos de estado:

```
GET /api/me          401 Unauthorized
GET /api/tenants     401 Unauthorized
GET /api/products    401 Unauthorized
```

Posteriormente, tras completarse la recarga de la aplicación, el ciclo de peticiones se estabilizó de manera limpia, manteniendo únicamente la solicitud primaria de validación de identidad:

```
GET /api/me -> 401 Unauthorized
```

Este comportamiento convalida que la arquitectura del cliente HTTP no induce al aplicativo a un bucle o ciclo infinito de solicitudes concurrentes ante un estado de falla por autenticación.

Asimismo, los registros capturados en la consola confirmaron la correcta propagación del evento a nivel interno:

```
[API Error]
[Auth] Session expired detected
```

5. Verificación de Código Fuente

Se realizó la auditoría de los componentes core que gobiernan el ciclo de vida de la autenticación:

a) Cliente HTTP centralizado (**lib/api.ts**)

Se constató que la totalidad de las peticiones salientes son gestionadas por una instancia única de Axios. Este cliente encapsula las respuestas nativas y mapea de forma homogénea los errores HTTP hacia estructuras controladas de tipo `ApiClientError`, registrando las anomalías mediante flujos estandarizados de diagnóstico:

```
console.error("[API Error]", error);
```

b) Lógica de Autenticación (**lib/auth.ts**)

Se evaluó la implementación del método crítico `getMe()`. El flujo intercepta con precisión los códigos de respuesta 401 emitidos por el backend, disparando el log de control pertinente y resolviendo la promesa con un valor nulo para sanitizar el estado del cliente:

```
// Ejemplo de lógica de control implementada en getMe
async function getMe() {
  try {
    const response = await api.get('/me');
    return response.data;
  } catch (error) {
    if (error.status === 401) {
      console.warn("[Auth] Session expired detected");
      return null;
    }
    throw error;
  }
}
```

c) Componente de Sincronización (**SyncAuth**)

Se comprobó la existencia de un mecanismo reactivo que evalúa el resultado de la identidad del usuario. Si el usuario retornado es válido, el estado global se actualiza a `authenticated`. En caso de recibir un valor nulo debido a una sesión expirada, el estado conmuta de inmediato a `unauthenticated`, orquestando de manera limpia la reactividad del frontend.

d) Middleware de Control de Acceso (**ProtectedRoute**)

Este componente actúa como la barrera de seguridad perimetral en las rutas del frontend. Mientras el estado de la sesión se encuentra indeterminado, preserva una bandera de `loading`. Si se confirma que el estado ha mutado a inválido, intercepta la navegación y ejecuta una redirección mandatoria hacia el path `/login`, bloqueando el renderizado de cualquier vista privada.

e) Almacenamiento de Estado Global (**auth.store.ts**)

Se auditó el almacén de estado (Store) encargado de centralizar la sesión. Se verificó que gestiona correctamente las transiciones de inicio de sesión (*login*), cierre de sesión (*logout*), propagación de errores y el refresco asíncrono de los metadatos del usuario autenticado.

6. Resultado del Análisis Técnico

Tras cruzar el comportamiento empírico de la aplicación con la lógica imperativa declarada en el código fuente, se determinó formalmente lo siguiente:

- La arquitectura de autenticación detecta con precisión matemática la pérdida de vigencia de las credenciales de sesión.

- El backend NestJS cumple rigurosamente con los estándares REST al rechazar el acceso con un código HTTP 401.
- El cliente React altera su máquina de estados global a un modo desautenticado de forma síncrona con la respuesta de la API.
- El sistema expulsa al usuario de las zonas restringidas y lo sitúa en la interfaz de Login de manera automatizada.
- Se descarta de forma categórica la existencia de loops o recursiones infinitas en las llamadas de red.

Por lo tanto, **el comportamiento observado mapea con el diseño esperado de un sistema SaaS** con autenticación robusta basada en sesiones.

7. Oportunidades de mejora

A pesar de que el sistema cumple con las directrices de seguridad y estabilidad, se han estructurado las siguientes iniciativas para refinar la experiencia de usuario (UX) y el manejo preventivo de excepciones:

- **Notificación Proactiva al Usuario:** Implementar un banner o notificación de tipo Toast antes de la redirección definitiva, informando explícitamente al operador que su sesión ha expirado por inactividad o revocación.
- **Centralización Absoluta de Errores 401:** Desplegar un interceptor global de Axios que capture los errores de autorización de manera centralizada, evitando que los métodos de servicio individuales requieran bloques repetitivos de control.
- **Filtro y Granularidad de Estados de Sesión:** Incrementar la granularidad en la capa de autenticación para discernir de forma explícita si la invalidez obedece a una *sesión expirada por tiempo*, una *sesión inexistente*, una *sesión explícitamente revocada* o un *error general de credenciales*.
- **Humanización de Mensajes:** Sustituir los registros crudos de consola de desarrollo por trazas y logs estructurados bajo un entorno controlado de telemetría de errores.

8. Validación de Criterios de Aceptación

Criterio de Verificación	Resultado de la Evaluación
API responde con código HTTP 401 ante tokens o sesiones inválidas	✓ Correcto (Funcionamiento según Diseño)
Frontend intercepta y detecta la invalidez de la sesión	✓ Correcto (Mapeo Exitoso)
El estado global de autenticación conmuta a "unauthenticated"	✓ Correcto (Transición Limpia)
Redirección automatizada del usuario hacia la interfaz de Login	✓ Correcto (Protección de Ruta Activa)
Los recursos protegidos permanecen totalmente inaccesibles	✓ Correcto (Aislamiento de Rutas)
Ausencia de bucles infinitos de peticiones concurrentes a la API	✓ Correcto (Estabilidad de Red)

9. Conclusión

La evaluación exhaustiva determinó que el mecanismo de autenticación implementado en la plataforma KiosQ mitiga correctamente los riesgos de acceso y responde con resiliencia ante la caducidad de una sesión. La plataforma salvaguarda los recursos restringidos mediante códigos HTTP 401 normativos, actualiza instantáneamente el contexto del cliente y fuerza la re-autenticación del usuario sin comprometer el rendimiento general.

Las observaciones recopiladas se han tipificado como optimizaciones enfocadas en la experiencia de usuario y en la modularidad del código fuente, concluyendo formalmente que **no existen vulnerabilidades o fallos críticos (bugs) en el núcleo de este flujo**.

Dictamen Técnico del Equipo de Ingeniería (CSS)

Con base en el análisis estático y dinámico del sistema, es técnicamente incorrecto catalogar este comportamiento como un defecto (bug) de autenticación, dado que el flujo de control opera exactamente conforme a los requerimientos de diseño de una arquitectura SaaS segura. Esta actividad queda oficialmente registrada como una verificación y análisis de incidentes de expiración de sesión, sirviendo como base de conocimiento técnico y documentación de buenas prácticas de código para la plataforma.